

From Tables to Notebooks

Now the data is on your machine. Time to start the real work — one notebook at a time.

Before you start

- If you do not understand something, that is normal — search it, read the docs, ask ChatGPT or any AI assistant, ask in the classroom
- Do not copy code you cannot explain. If you use AI, make sure you understand every line before you keep it

Where we are now

- ✓ The database is running locally with the Olist tables inside
- ✓ You can query the tables and join them
- ✓ You understand the tables, the relations, and the problem: predict late vs on-time delivery

What is an artifact?

- An artifact is any file your notebook saves so the next step can use it without running your notebook again
- Examples: the joined ML table, the labeled table, the train/val/test files, saved charts, a statistics summary, the feature list, the trained model
- Rule: every notebook reads artifacts from the step before it, and writes its own artifacts for the step after it
- We also save them so we can reproduce the same result later — when the production pipeline runs, its output must match what the notebook produced
- Later in the track we will turn these notebooks into production Python scripts — so keep them clean and in order

The notebooks

1

Notebook 1 — Read & join the tables

- Read every table from the database and look at each one on its own
- Check row counts, keys, duplicates, and what one row means in each table
- **Aggregate before you join — order items and payments have many rows per order**
- Keep the analysis light here: only what you need to join correctly
- **Artifact: one ML table, one row per order**

2

Notebook 2 — Create the labels

- Build the label from the delivery date against the estimated delivery date
- Check the label is correct on a few real orders before you trust it
- Look at the class distribution — how many late, how many on time
- **Decide if you have a class imbalance problem, and how big it is**
- **Artifact: the labeled table**

3

Notebook 3 — Train / validation / test split

- **Split the data before any deep analysis, to avoid leaking the test set into your decisions**
- Think about how to split: random or by time, and why
- Small analysis to help you choose — check the date range and the label balance in each split
- Keep the same label ratio across the splits if you split randomly
- **Artifact: train, validation, and test files**

4

Notebook 4 — EDA (the detailed one)

- **Work on the training split only — do not open the test set**
- Data types, shape, and memory: what is numerical, categorical, date, text, ID
- Missing values: how many, where, and is the missing itself meaningful
- Numerical: statistics, distributions, skew, outliers, ranges that make no sense
- Categorical: value counts, cardinality, rare categories, messy or duplicated values
- Relations: groupby against the label, correlations, cross-tabs, charts
- Dates: order dates, delivery times, seasonality, weekday and holiday effects
- Geography: customer and seller state, distance, ZIP-code prefix
- **Write down what you find — the goal is to decide your features and your model**
- **Artifact: saved charts and a short findings summary**

5

Notebook 5 — Feature engineering

- Build the features you decided on in the EDA notebook
- Handle missing values, encode categories, scale if your model needs it
- **Use only information available at prediction time — no future columns**
- **Fit any transformation on the training split, then apply it to validation and test**
- **Save every fitted object you used — encoders, scalers, imputers — not just the output table**
- **In production the pipeline must load the same fitted objects, never fit again on new data**
- **Artifact: the final feature table, the fitted transformers, and the feature list**

6

Notebook 6 — Train, tune, evaluate

- **Start with a simple baseline so you know what you have to beat**
- Train a model, then tune it using the validation split
- **Choose a metric that fits an imbalanced problem — not accuracy alone**
- **Touch the test set once, at the very end**
- **Artifact: the trained model and a results summary**

**Done when**

- ✓ Six notebooks, each doing one job, running in order from a clean start
- ✓ Every notebook saves its artifacts, and reads the artifacts of the step before it
- ✓ You can explain why you split the data the way you did
- ✓ You have a first model result compared against a simple baseline

Notes

- One notebook, one job — do not put everything in one file
- Keep the notebooks numbered so the order is obvious
- No production scripts yet — that comes in a later task
- Take your time on Notebook 4, everything after it depends on what you find there

Good luck guys!

من الجداول إلى النوتبوكس

هلاً البيانات صارت عندك على جهازك. وقت نبش الشغل الحقيقي — نوتبوك ورا نوتبوك.

قبل ما تبش

- إذا في إشي ما فهمته، هذا شي طبيعي — دور عليه، اقرأ الدوكيومنتيشن، أسأل ChatGPT أو أي مساعد ذكاء اصطناعي، أو أسأل بالكلاس
- لا تنسخ كود ما بتقدر تشرحه، إذا استعملت AI، تأكد إنك فاهم كل سطر قبل ما تخليه

وين صرنا هلاً

- ✓ قاعدة البيانات شغالة محلياً وجداول Olist جواتها
- ✓ تقدر تعمل query و join على الجداول
- ✓ فاهم الجداول والعلاقات والمشكلة: نتوقع إذا الطلب رح يتأخر أو يوصل بالوقت

شو يعني artifact ؟

- ال artifact هو أي ملف بيحفظه النوتبوك حتى الخطوة اللي بعده تستعمله بدون ما تشغل النوتبوك من جديد
- أمثلة: جدول ال ML بعد الدمج، الجدول مع الليبل، ملفات train و validation و test ، الرسومات المحفوظة، ملخص الإحصائيات، قائمة الفيتشرز، الموديل المدرب
- القاعدة: كل نوتبوك يقرأ artifacts من الخطوة اللي قبله، ويكتب artifacts للخطوة اللي بعده
- كمان بنحفظها حتى نقدر نعيد إنتاج نفس النتيجة لاحقاً — لما يشتغل بايلاين الإنتاج، لازم مخرجاته تطابق اللي طلع من النوتبوك
- لاحقاً بالتراك رح نحول هالنوتبوكس لسكربتات بايثون جاهزة للإنتاج — فخليها مرتبة ونظيفة

النوتبوكس

نوتبوك 1 — قراءة الجداول ودمجها

1

- اقرأ كل جدول من قاعدة البيانات وشوف كل واحد لحاله
- افحص عدد الصفوف، المفاتيح، التكرارات، وشو يمثّل الصف الواحد بكل جدول
- اعمل aggregation قبل ال join — جداول ال items وال payments فيها أكثر من صف للطلب الواحد
- خلي التحليل هون خفيف: بس اللي بتحتاجه عشان تدمج صح
- ال artifact : جدول ML واحد، صف واحد لكل طلب

نوتبوك 2 — إنشاء الليبل

2

- اعمل الليبل من تاريخ التوصيل مقارنة بالتاريخ المتوقع للتوصيل
- تأكد إن الليبل صحيح على كم طلب حقيقي قبل ما تعتمد عليه
- شوف توزيع الكلاسات — كم طلب متأخر وكم وصل بالوقت
- قرر إذا عندك مشكلة class imbalance وقديش حجمها
- ال artifact : الجدول مع الليبل

نوتبوك 3 — تقسيم train و validation و test

3

- قسم البيانات قبل أي تحليل عميق، حتى ما تسرب الـ test لقراراتك
- فكر كيف بدك تقسم: عشوائي أو حسب الوقت، وليش
- تحليل بسيط يساعدك تقرر — شوف مدى التواريخ وتوازن الليبل بكل قسم
- حافظ على نفس نسبة الليبل بين الأقسام إذا قسمت عشوائي
- الـ artifact : ملفات train و validation و test

نوتبوك 4 — الـ EDA (التفصيلي)

4

- اشتغل على قسم الـ training بس — لا تفتح الـ test
- أنواع البيانات والحجم: شو رقمي، شو تصنيفي، شو تاريخ، شو نص، شو ID
- القيم الناقصة: قديمش، وين، وهل النقص نفسه بيحمل معنى
- الرقمي: إحصائيات، توزيعات، skew، قيم شاذة، ومدايات ما إلها معنى
- التصنيفي: عدد القيم، الـ cardinality، الفئات النادرة، والقيم المكررة أو الفوضوية
- العلاقات: groupby مقابل الليبل، الارتباطات، الجداول المتقاطعة، والرسومات
- التواريخ: تواريخ الطلبات، مدة التوصيل، الموسمية، وتأثير أيام الأسبوع والعطل
- الجغرافيا: ولاية العميل والبائع، المسافة، وبادئة الرمز البريدي
- سجل اللي بتلاقيه — الهدف إنك تقرر الفيتشرز والموديل
- الـ artifact : الرسومات المحفوظة وملخص قصير للنتائج

نوتبوك 5 — هندسة الفيتشرز

5

- ابن الفيتشرز اللي قررتها بنوتبوك الـ EDA
- تعامل مع القيم الناقصة، رمز الفئات، وطبق scaling إذا الموديل يحتاجه
- استعمل بس المعلومات المتوفرة وقت التوقع — بدون أعمدة من المستقبل
- درب أي تحويل على قسم الـ training بس، وبعدها طبقه على validation و test
- احفظ كل object دربته — encoders و scalers و imputers — مو بس الجدول الناتج
- بالإنتاج لازم البايبلان يحمل نفس الـ objects المحفوظة، وما يعيد التدريب على بيانات جديدة
- الـ artifact : جدول الفيتشرز النهائي والتحويلات المدربة وقائمة الفيتشرز

نوتبوك 6 — التدريب والضبط والتقييم

6

- ابدأ بـ baseline بسيط حتى تعرف شو لازم تتغلب عليه
- درب موديل، وبعدين اضبطه باستعمال قسم الـ validation
- اختار metric بناسب مشكلة غير متوازنة — مو الـ accuracy لحاله
- لا تفتح الـ test إلا مرة وحدة، بالنهاية
- الـ artifact : الموديل المدرب وملخص النتائج

التاسك خالص لما

- ✓ ست نوت بوكس، كل واحد بعمل شغلة وحدة، ويشتغلوا بالترتيب من بداية نظيفة
- ✓ كل نوت بوك يحفظ ال artifacts تبعته، ويقرأ artifacts الخطوة اللي قبله
- ✓ تقدر تشرح ليش قسمت البيانات بالطريقة
- ✓ عندك أول نتيجة موديل مقارنة مع baseline بسيط

ملاحظات

- نوت بوك واحد، شغلة وحدة — لا تحط كل إشي بملف واحد
- رقم النوت بوكس حتى الترتيب يكون واضح
- ما في سكربتات إنتاج هلاً — رح تيجي بتاسك لاحق
- خذ وقتك بنوت بوك 4، كل اللي بعده يعتمد على اللي بتلاقيه فيه

بالتوفيق للجميع!